Rakshaka AI: Model Report

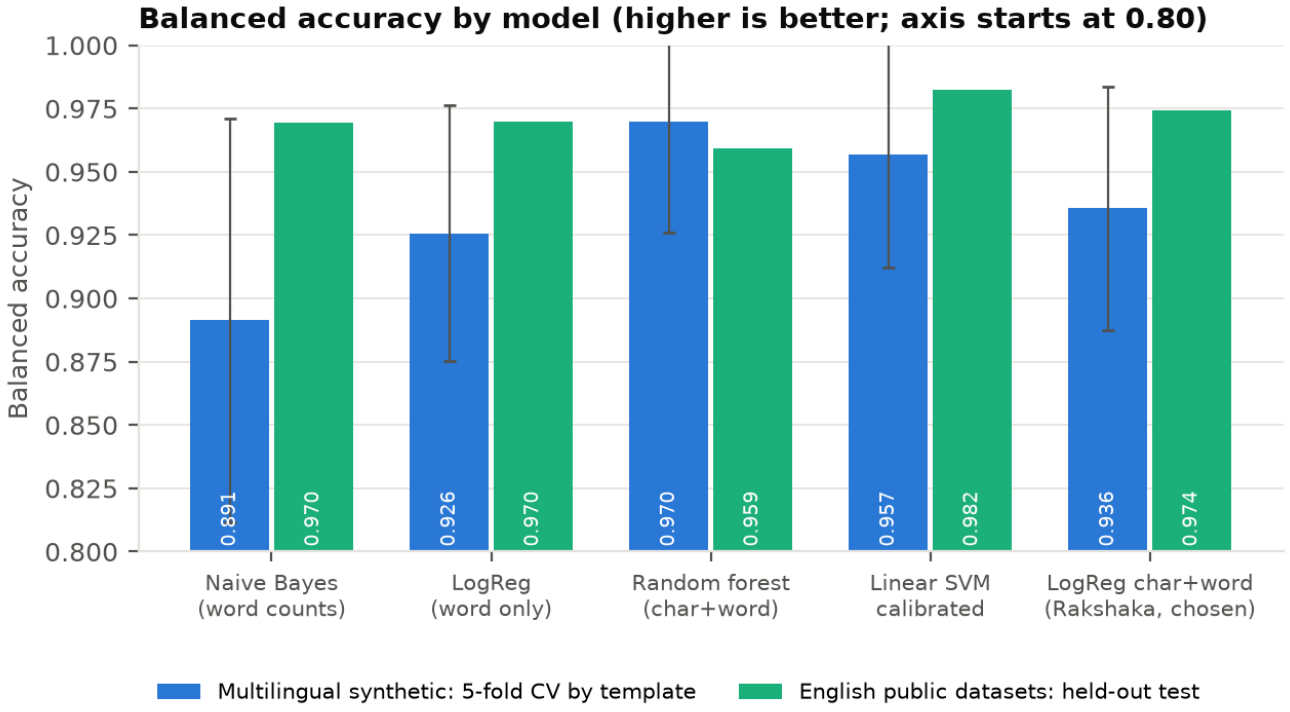
Scam-message classifier for Sri Lanka (English, Sinhala, Singlish, Tamil, Tanglish). Model v1 (clean: 0 human-review labels), trained 2026-10-17. Report built 17 October 2026.

Held-out test	Scams missed	Legit flagged	Shifted-set drift
0.926 balanced accuracy 0.946 macro F1	**0** of 190	**11** of 74	**Fires**: unknown-token rate 38.1% vs alert line 30.7%

All scores come from **synthetic** messages written for this project (no real personal data). The test set holds out whole message templates the model never saw, so it measures generalisation to new wording within known scam tactics. It is not evidence of real-world accuracy.

1. Why logistic regression on character + word n-grams

We compared five text classifiers on the same data, using the same preprocessing. Each was scored two ways: 5-fold cross-validation on the multilingual synthetic data, where whole templates are held out per fold, and a held-out test on the two English public datasets. **Our model is not the most accurate one we tested.** Random forest scored highest on the multilingual cross-validation. We chose logistic regression for the reasons below, which come from how the product works, not only from accuracy.



Model	CV balanced acc. (multilingual)	English test bal. acc.	Shifted-set bal. acc.	Confidence: normal to shifted	Speed (ms / message)
Naive Bayes (word counts)	0.891 ± 0.080	0.970	0.884	1.00 to 0.99	0.11
LogReg, word TF-IDF only	0.926 ± 0.050	0.970	0.977	0.75 to 0.73	0.21
Random forest (char+word)	0.970 ± 0.044	0.959	0.958	0.91 to 0.87	25.51
Linear SVM, calibrated (char+word)	0.957 ± 0.044	0.982	0.992	0.97 to 0.92	1.31
LogReg char+word (Rakshaka)	0.936 ± 0.048	0.974	0.973	0.83 to 0.80	0.57

Why we kept logistic regression

- **Fast enough to run on every message.** 0.57 ms per message on a laptop CPU, compared with 25.5 ms for the random forest (about 45 times slower). A bank help desk or SMS gateway needs answers in real time on cheap hardware.

- **Real probabilities, built in.** The API routes a message to a human when confidence is below 0.60, and the drift monitor watches mean confidence. Logistic regression outputs a probability directly. A linear SVM only gives probabilities through an extra calibration step, which trains three more models and slows the in-app */retrain* loop.
- **Consistent on both data sources.** It is second-best on the English public datasets (0.974). The random forest, which looks best on the multilingual templates, is the *worst* on the English public data (0.959) and loses accuracy on the shifted set (0.958). That pattern suggests it memorises template wording.
- **Character n-grams handle Sinhala, Tamil and romanised spelling.** Splitting Sinhala and Tamil script into words is unreliable, and Singlish and Tanglish have no fixed spelling. Character 2-5-grams cope with both. Adding them lifts cross-validation accuracy from 0.926 (words only) to 0.936, and test accuracy on Sinhala from 0.50 to 0.63.
- **Simple, explainable and quick to retrain.** One linear weight per n-gram, so a reviewer can be shown why a message was flagged. Retraining takes under a second, which keeps the human-review loop practical.

What we did not choose, and why

- **Keyword rules.** A hand-written keyword list (link, phone, OTP, PIN, prize, verify ...) reached only 0.724 balanced accuracy on the test set and cannot learn new tactics.
- **Naive Bayes.** The fastest model, but the least accurate on the multilingual data (0.891). Its confidence stays at about 0.99 even on new tactics, so a confidence-based drift signal would never fire.
- **Multilingual transformers (mBERT, XLM-RoBERTa) and LLM APIs.** *Not benchmarked.* In a 6-hour, CPU-only build they mean large downloads, slower inference and fine-tuning on about 1,400 synthetic messages, which tends to overfit. An LLM API would also send message text to a third party.

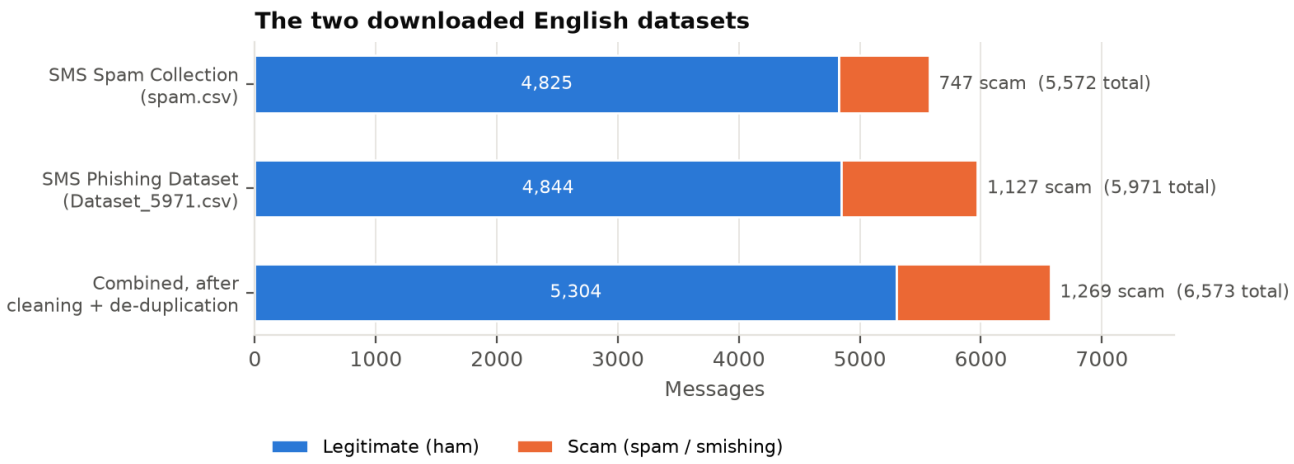
Recommended next experiment: the calibrated linear SVM had the best English-test score (0.982) and the best shifted-set score. With more real, reviewed data it is the first candidate to replace logistic regression, if its calibration holds up.

2. Data from the two online datasets

Two public English SMS datasets were downloaded during the build window:

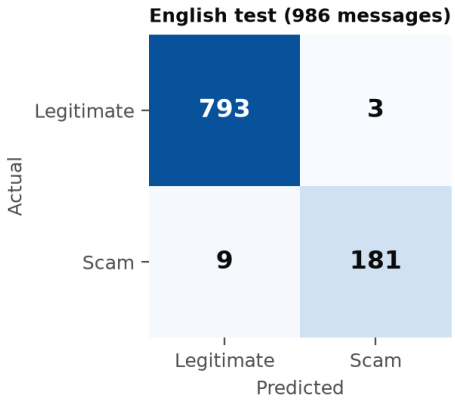
Dataset	What it contains	Raw labels	Use in Rakshaka
SMS Spam Collection (Almeida & Gómez Hidalgo, UCI Machine Learning Repository; Kaggle copy *spam.csv*, columns v1/v2)	5,572 English SMS messages	4,825 ham, 747 spam	English baseline pipeline (scripts/), model comparison
SMS Phishing Dataset for Machine Learning and Pattern Recognition (Mendeley Data; *Dataset_5971.csv*, columns LABEL, TEXT, URL, EMAIL, PHONE)	5,971 English SMS messages	4,844 ham, 638 smishing, 489 spam (mixed case)	English baseline pipeline (scripts/), model comparison

Licences still need to be confirmed and recorded in the README. Because redistribution rights are not established, the raw files are **not** committed to the repository (.gitignore).



How the two datasets were combined

- **Read safely.** spam.csv is not valid UTF-8. Its 1,182 invalid bytes were decoded as Windows-1252, which recovers characters such as the pound sign. Dataset_5971 contains 185 replacement characters whose original text cannot be recovered; they were kept as they are.
- **Map labels to two classes.** *ham* becomes *legitimate*; *spam* and *smishing* become *scam*. Case variants such as 'Smishing' and 'smishing' are merged.
- **Normalise and de-duplicate.** Unicode NFKC normalisation, collapsed whitespace, and a SHA-256 *group id* for the case-folded text. 4,970 duplicate rows were merged, leaving **6,573** unique messages. A message that appears in both datasets with different labels would stop the pipeline; this did not happen.
- **Keep provenance.** Every row keeps its original labels and its source rows, for example *spam.csv:120|Dataset_5971.csv:88*, plus URL, email and phone flags.
- **Split by group.** 70 / 15 / 15 train / validation / test, by group id and stratified by label, so the same text never appears in two splits. Only the training set is balanced, by downsampling legitimate groups.



English baseline result (scripts/train_model.py)

Test: 986 unique held-out messages. Balanced accuracy **0.974**, macro F1 **0.980**. 9 scams missed and 3 legitimate messages flagged.

This model is English-only. The deployed multilingual model (sections 3-5) is trained on the synthetic multilingual data instead. The English datasets serve as an independent second benchmark, and as the source for a human translation sample (below).

Human translation path (prepared, not yet used)

scripts/prepare_data.py draws a balanced sample of 200 English messages (100 per class) into *data/annotation/translation_sample.csv*, with columns for Sinhala, Singlish, Tamil and Tanglish. A translation is used for training only after a native-speaker reviewer marks it *verified* and adds their name. The code never translates automatically. **So far 0 translations are verified.**

3. How the Sinhala, Singlish, Tamil and Tanglish data was created

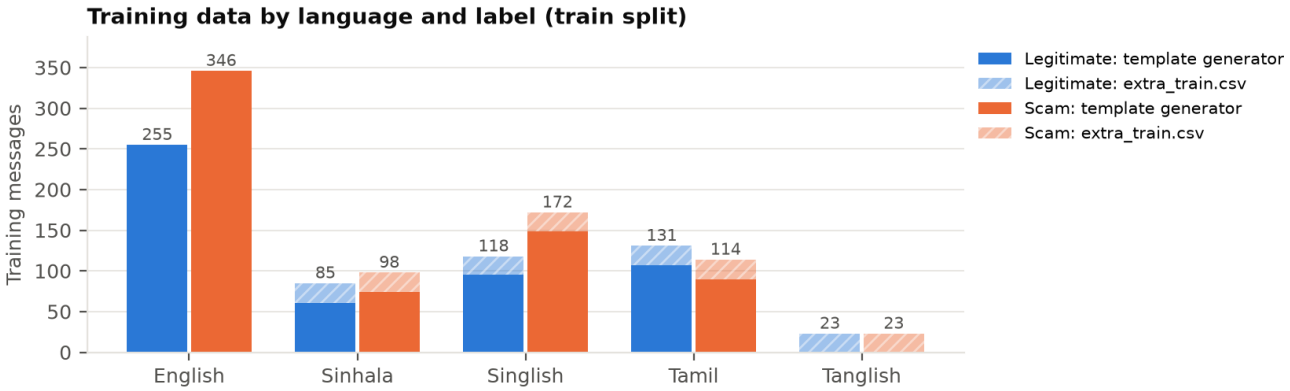
No public Sinhala, Singlish, Tamil or Tanglish scam-SMS dataset with a clear licence was available. We also may not use real people's messages (PDPA No. 9 of 2022). So all multilingual data is **synthetic**, from two sources:

- **Template generator** (*src/make_data.py*, seed 42). 34 scam and 34 legitimate message templates, hand-written by the team with AI assistance and modelled on common Sri Lankan scam tactics: blocked bank accounts, OTP and PIN requests, lottery and parcel fees, fake jobs, Aswesuma payments, pensions, loans, electricity disconnection. Each template is filled 70 times with fake slot values: DemoBank, Rs. amounts, *.example links, 07X-XXXXXXX numbers, fictional names and times. Some copies get random lower case, upper case or '!!' noise.
- **Hand-curated extra set** (*data/extra_train.csv*, 188 messages), written with Claude Code: 24 + 24 Sinhala, 24 + 24 Tamil, 23 + 23 Singlish and 23 + 23 Tanglish (scam + legitimate). It adds new tactics (fake police, account-takeover codes, CVV requests, scholarship fees) and safe bank-style messages, such as an OTP saying 'do not share'. It also adds Tanglish, which the generator does not cover. No row copies or translates a held-out test or validation template, or a shifted-set tactic; the closest text match is 62% similar. **These messages have not yet been checked by native speakers.**

Language	Script	Templates (scam + legit)	Example (all synthetic)
English	Latin	31	URGENT: Your DemoBank account is blocked. Verify now at secure-verify.example
Sinhala	Sinhala	12	(Sinhala script) 'Your DemoBank account has been suspended. Confirm immediately via [link].'

Language	Script	Templates (scam + legit)	Example (all synthetic)
Singlish	Latin (romanised Sinhala)	13	Machan oyage DemoBank account eka block wela. Me link eken verify karanna ...
Tamil	Tamil	12	(Tamil script) 'Your DemoBank account has been frozen. Verify at [link] immediately.'
Tanglish	Latin (romanised Tamil)	0 (extra set only)	Ungal current bill kattavillai. Indru iravu connection cut seiyappadum ...

Sinhala and Tamil examples are shown translated, because the PDF fonts cannot render those scripts.



Splitting without leaks

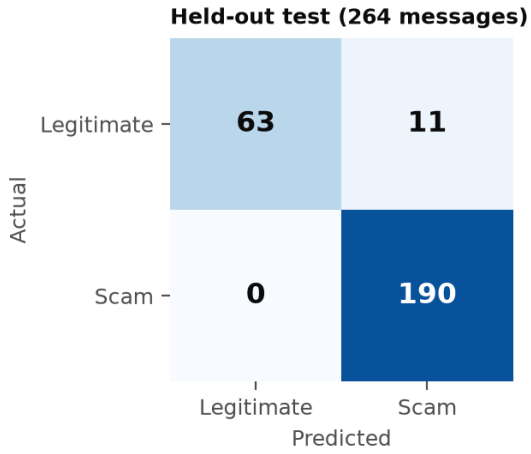
Messages generated from the same template share almost all their words. If copies of one template landed in both train and test, the test score would be inflated. So the generator holds out **whole templates**, stratified by (label, language):

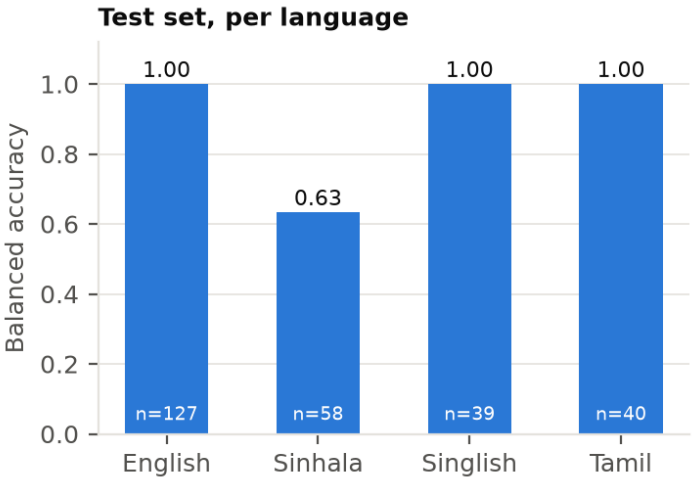
Split	Messages	Templates / groups	Purpose
Train	1,365	234	Fit the model (includes the 188 extra_train rows, one group each)
Validation	276	10	Measure the drift baselines only (confidence, unknown-token rate)
Test (held out)	264	12	Reported metrics: unseen wording, known tactics
Shifted	152	15	Drift demo: new tactics (SL-UDI digital ID / QR codes, biometric fees, USDT / AI crypto, 'new number' impersonation, voice notes, heavier code-mixing)

4. How the model is trained

Step	What happens	Where
1. Generate	Write train / val / test / shifted CSVs from the templates and append data/extra_train.csv to train.	src/make_data.py
2. Pre-process	NFKC normalisation, case-folding, whitespace collapse. Add marker tokens *urltoken*, *emailtoken* and *phonetoken* when a link, email address or phone number is present.	rakshaka/modeling.py
3. Features	Union of two TF-IDF vectorisers (sublinear tf, min_df 2): character 2-5-grams (up to 250k features) and word 1-2-grams (up to 100k).	make_classifier()
4. Classifier	Logistic regression, class_weight='balanced' because scam outnumbers legitimate about 55 / 45, max_iter 1500, random_state 42. Human labels from data/reviewed.csv are added when present.	src/train.py
5. Drift baselines	Score the unseen validation templates. Mean confidence 0.828, unknown-token rate 18.7%, unseen char-n-gram rate 24.3%. Saved with the word vocabulary in models/meta.json and vocab.json.	src/train.py
6. Evaluate	Held-out test and shifted set: balanced accuracy, macro F1, confusion matrix, per-language scores, confidence and drift rates. Written to reports/metrics.json.	src/evaluate.py
7. Serve and retrain	FastAPI loads model.joblib. Reviewed labels go to data/reviewed.csv; *POST /retrain* runs step 4-5 again and hot-loads the new version (v1, v2, ...).	src/api.py

5. Results on the held-out test set

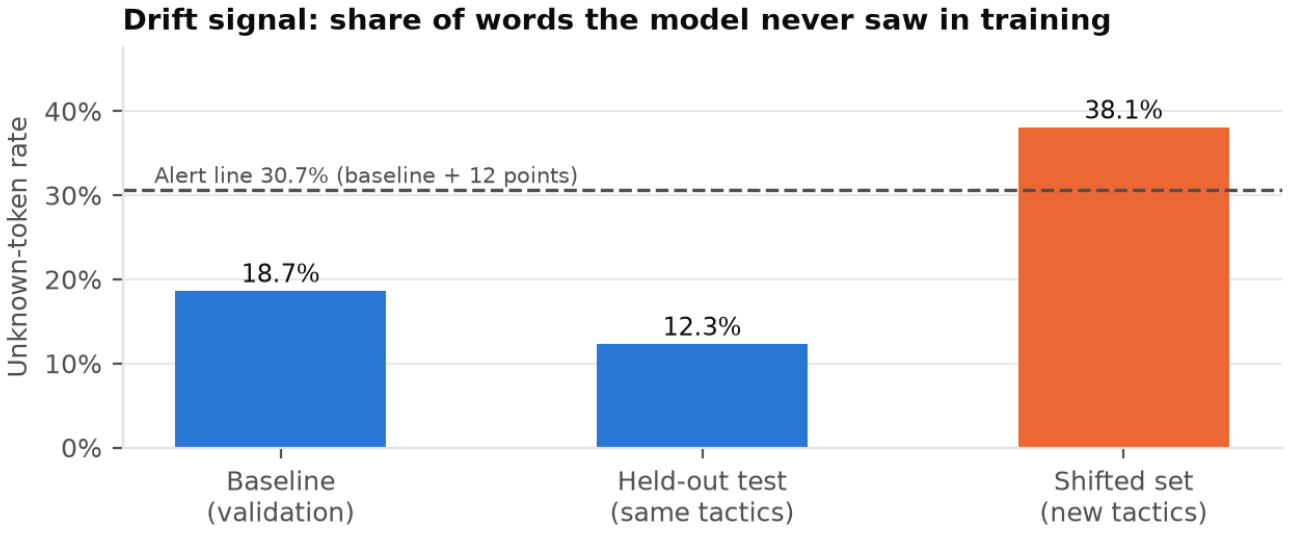




Class	Precision	Recall	F1	Messages
Legitimate	1.000	0.851	0.920	74
Scam	0.945	1.000	0.972	190

- **No scam was missed** (recall 1.00). 11 of 74 legitimate messages were flagged as scam. For a scam checker that is the safer kind of mistake.
- **Sinhala is the weak spot** (0.63). The held-out Sinhala legitimate template, a water-bill notice, is flagged as scam in 11 of 15 copies; these are all of the test set's false alarms. Adding the Sinhala extra set raised this from 0.50 to 0.63. More native-checked, legitimate Sinhala messages are the top data need.
- **Tanglish cannot be scored yet.** The test set has no Tanglish messages; a separately written Tanglish test set is needed.
- Shifted set (new tactics): balanced accuracy 1.000. The new tactics still use links and phone numbers, which the model already treats as scam signals. That is why the product uses an **input-drift** alarm rather than waiting for accuracy to fall.

6. Drift monitoring and response



The API keeps a rolling window of the last 50 predictions and judges it once it holds 20. It raises a **DRIFT** alert if the window's unknown-token rate rises more than 12 points above the validation baseline, or if mean confidence drops more than 0.10. New-tactic messages average 38.1% unknown words, compared with 18.7% at baseline, so the alert fires while accuracy is still high. The product notices the change before it starts failing silently.

- **Safe fallback:** during drift, and for any message with confidence below 0.60 or 50% or more unseen words, the verdict is 'UNCERTAIN - be careful, sent for review'. It is never 'safe'.
- **Human review, then retrain:** reviewers label queued messages in the dashboard. */retrain* builds the next version from those labels and hot-loads it. Labels must be set deliberately: wrong labels poison the next model.
- **Privacy:** predictions are logged with a text hash only. Message text is stored only for messages routed to human review.

7. Limitations and next steps

- All multilingual data is synthetic, and the test set holds only 12 templates. Scores have wide error bars (cross-validation ±0.05) and are not real-world accuracy.
- The Sinhala, Tamil, Singlish and Tanglish texts have not been checked by native speakers. Verify them, and complete the 200-message human translation sample.
- Write a Tanglish test set, and more legitimate Sinhala messages.
- Re-run this comparison with the calibrated linear SVM once real reviewed data exists.
- Confirm and record the licences of both English datasets in the README.